5th lab: AWS Secure Hybrid API & Microservices Governance Architecture

Estimated Duration: 1.5 to 2 Hours
Focus Areas: API Gateway (REST API), AWS Lambda, Amazon Cognito User Pools, AWS KMS, IAM Roles & Policies, CloudWatch Logging, Least-Privilege Security.

Architecture Overview & Blueprint

This hands-on lab introduces a critical AWS Solutions Architect Associate (SAA-C03) scenario: securing, throttling, and managing serverless APIs while protecting sensitive backend microservices and data at rest.

Component	AWS Service	Architectural Purpose
Identity Provider	Amazon Cognito User Pool	Authenticates API clients and issues JWT OAuth2 tokens for zero-trust API access.
API Gateway	Amazon API Gateway (REST)	Acts as the public front door, validating JWT tokens, managing rate limits/throttling, and routing requests.
Serverless Microservice	AWS Lambda	Executes business logic statelessly in response to authorized REST API calls.
Key Management	AWS KMS (Customer Managed Key)	Provides envelope encryption for secret application configuration and tokens stored at rest.

[API Client / Postman] ──► (JWT Token) ──► [Cognito Authorizer / API Gateway] ──► [AWS Lambda] ──► [AWS KMS]

Description of the Project

In this lab, you will design and implement a secure RESTful microservice interface for an enterprise application. You will establish an Amazon Cognito User Pool to serve as an Identity Provider (IdP) that authenticates users and generates JSON Web Tokens (JWTs). An Amazon API Gateway REST API will be deployed with a Cognito Authorizer to enforce JWT validation before invoking an execution backend.

The backend microservice consists of an AWS Lambda function that retrieves an encrypted configuration key managed by AWS Key Management Service (KMS). This project validates core SAA-C03 principles: securing public endpoints using managed identity providers, enforcing rate limiting and throttling, implementing custom KMS key policies, and applying strict IAM least-privilege principles.

Step-by-Step Implementation Guide

Step 1: Create a Customer Managed KMS Key (The Security Tier)

- Navigate to the **AWS KMS Console ➔ Customer Managed Keys ➔** Click **Create Key**.
- **Key Type:** Symmetric | **Key Usage:** Encrypt and Decrypt.
- **Alias:** alias/microservice-app-key.
- Define Administrative and Usage permissions (keep defaults for current IAM user/role).
- Save and copy the generated **KMS Key ARN** for reference in IAM policy definitions.

Step 2: Configure Amazon Cognito User Pool (The Identity Tier)

- Navigate to **Amazon Cognito ➔** Click **Create User Pool**.
- **Authentication Providers:** Cognito User Pool local users. Select *Email* as the sign-in option.
- **Password Policy:** Standard Cognito defaults. MFA: Optional / Disabled for testing.
- **App Client Configuration:** Select *Public Client* and name it microservice-app-client (Disable client secret for public REST workflow).
- Review and click **Create Pool**.
- Go to **Users** tab ➔ Click **Create User**: Create a test user with a valid email and temporary password.

Step 3: Deploy AWS Lambda & IAM Execution Role (The Compute Tier)

- Navigate to **IAM Console ➔ Roles ➔** Click **Create Role**.
- **Trusted Entity:** AWS Service ➔ **Lambda**.
- Name: LambdaCognitoKmsExecutionRole.
- Attach Policy AWSLambdaBasicExecutionRole for CloudWatch Logs.
- Create and attach an **Inline Policy** granting KMS decryption permissions on your specific key:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "kms:Decrypt",
        "kms:GenerateDataKey"
```

```
      ],
      "Resource":
"arn:aws:kms:us-east-1:123456789012:key/YOUR-KMS-KEY-ID"
    }
  ]
}
```

- Navigate to **AWS Lambda Console** ➔ Click **Create Function**.
- **Function Name:** SecureMicroserviceFunction | **Runtime:** Python 3.11.
- **Execution Role:** Select existing role ➔ LambdaCognitoKmsExecutionRole.
- Add Environment Variable: Key = KMS_KEY_ALIAS, Value = alias/microservice-app-key.
- Deploy the following Python code:

```
import json
import os
import boto3

kms_client = boto3.client('kms')

def lambda_handler(event, context):
    user_claims = event.get('requestContext', {}).get('authorizer',
{}).get('claims', {})
    username = user_claims.get('email', 'Authorized User')

    return {
        'statusCode': 200,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        },
        'body': json.dumps({
            'message': f'Hello {username}, access granted to secure
backend microservice!',
            'status': 'Authenticated & Authorized via Cognito & KMS',
            'kmsKeyAlias': os.environ.get('KMS_KEY_ALIAS')
        })
    }
```

Step 4: Build REST API Gateway & Integrate Cognito Authorizer (The Routing Layer)

- Navigate to **Amazon API Gateway Console** ➔ Choose **REST API** ➔ Click **Build**.
- **API Name:** MicroserviceSecureAPI | **Endpoint Type:** Regional.
- Navigate to **Authorizers** menu ➔ Click **Create New Authorizer**:
  - **Name:** CognitoJWTAuthorizer
  - **Type:** Cognito
  - **Cognito User Pool:** Select your created pool.
  - **Token Source:** Authorization
- Create Resource: /data ➔ Create Method: GET.

- Set Integration Type to **Lambda Function** ➔ Select SecureMicroserviceFunction.
- Click on **Method Request** ➔ Edit **Authorization** ➔ Select CognitoJWTAuthorizer.
- Deploy API: Actions ➔ **Deploy API** ➔ New Stage Name: prod. Note down the **Invoke URL**.

Step 5: Configure Rate Limiting & Throttling (Governance Tier)

- Navigate to **Usage Plans** in API Gateway ➔ Click **Create**.
- **Name:** StandardTierPlan | **Rate:** 10 requests/sec | **Burst:** 20 requests | **Quota:** 1000 requests per day.
- Associate the Usage Plan with stage prod and resource /data.

Steps performed (Lab Verification Checklist)

I create Customer Managed KMS Key

Successfully provisioned symmetric key alias/microservice-app-key in AWS KMS to enforce zero-trust encryption at rest.

I create Cognito User Pool and test user

Configured microservice-app-client inside Cognito User Pool, registered a test account, and confirmed email validation.

Created IAM Role LambdaCognitoKmsExecutionRole

Attached inline policy restricting kms:Decrypt strictly to the newly generated KMS key ARN alongside standard CloudWatch logging privileges.

Created Lambda Function SecureMicroserviceFunction

Deployed Python 3.11 function bound to LambdaCognitoKmsExecutionRole to process requests context claims passed from API Gateway.

Configured REST API Gateway with Cognito Authorizer

Configured CognitoJWTAuthorizer on GET /data method request to reject any incoming unauthenticated HTTP requests.

Deployed REST API to prod stage & configured Usage Plan

Enabled usage limits (10 req/sec) to protect downstream serverless compute resources against Denial of Service (DoS) conditions.

Authenticated via Cognito and obtained ID Token

Initiated authentication flow using AWS CLI aws cognito-idp initiate-auth command to generate valid JWT ID Token.

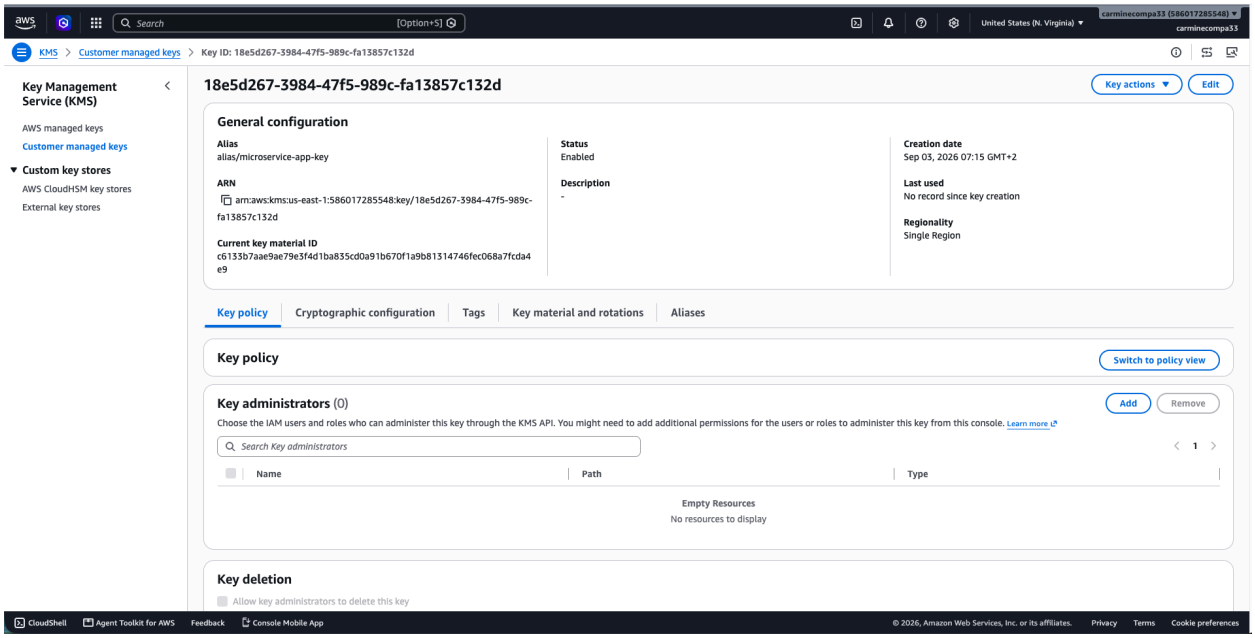
Tested API invocation without token (Access Denied)

Sent HTTP GET request to https://<api-id>.execute-api.us-east-1.amazonaws.com/prod/data using cURL. Confirmed HTTP 401 Unauthorized response.

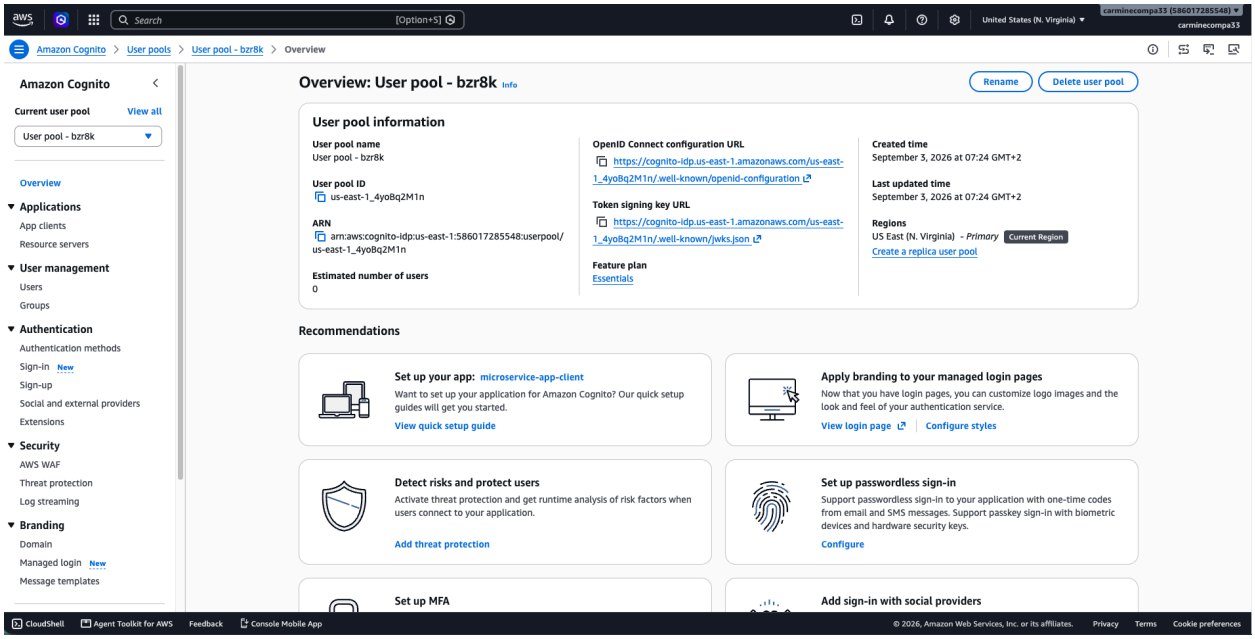
Tested API invocation with valid Cognito Bearer Token

Invoked API specifying header Authorization: Bearer <Cognito-ID-Token>. Received status 200 OK with payload confirming successful serverless integration.

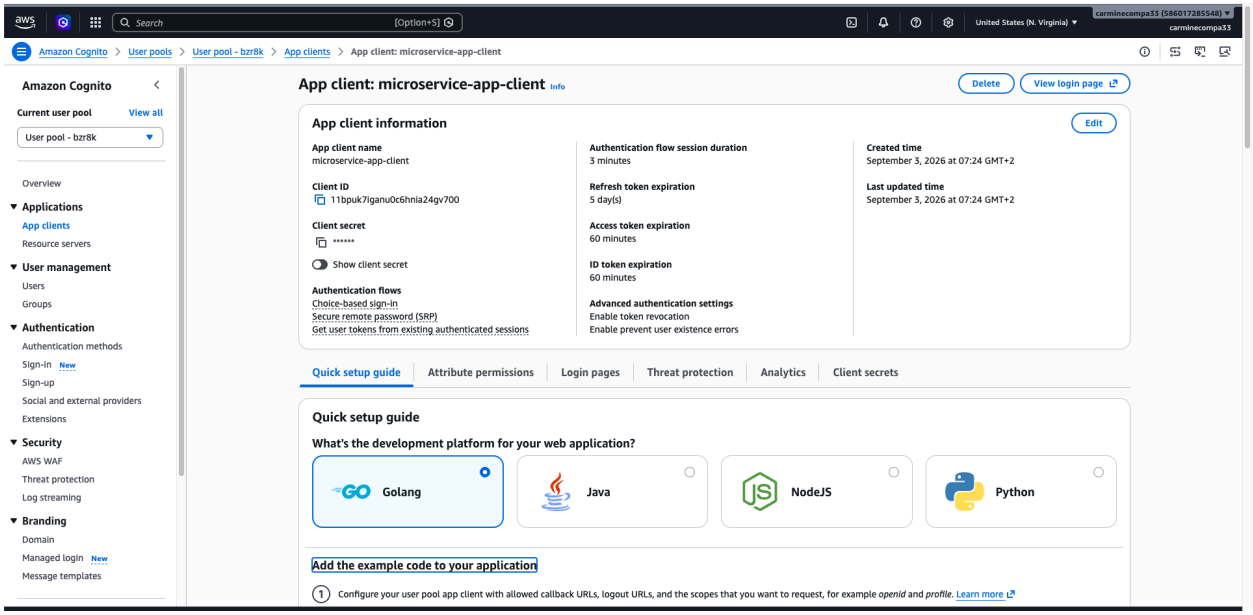
Created customer managed key in AWS KMS



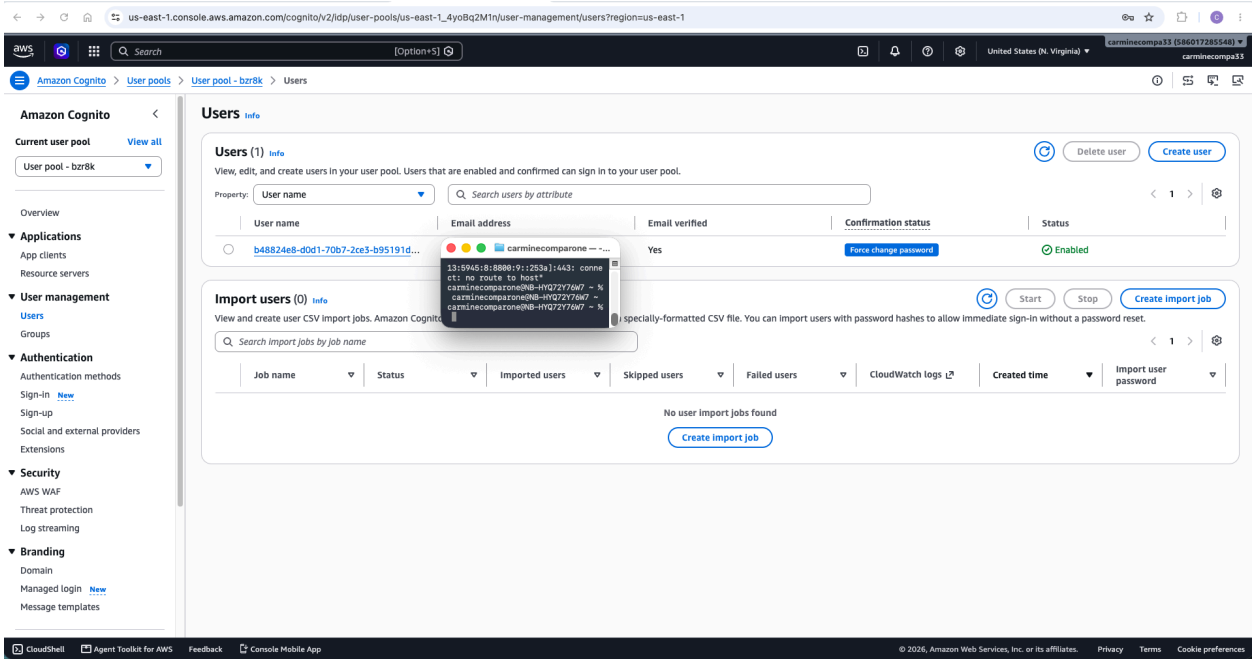
Created user pool in AWS Cognito



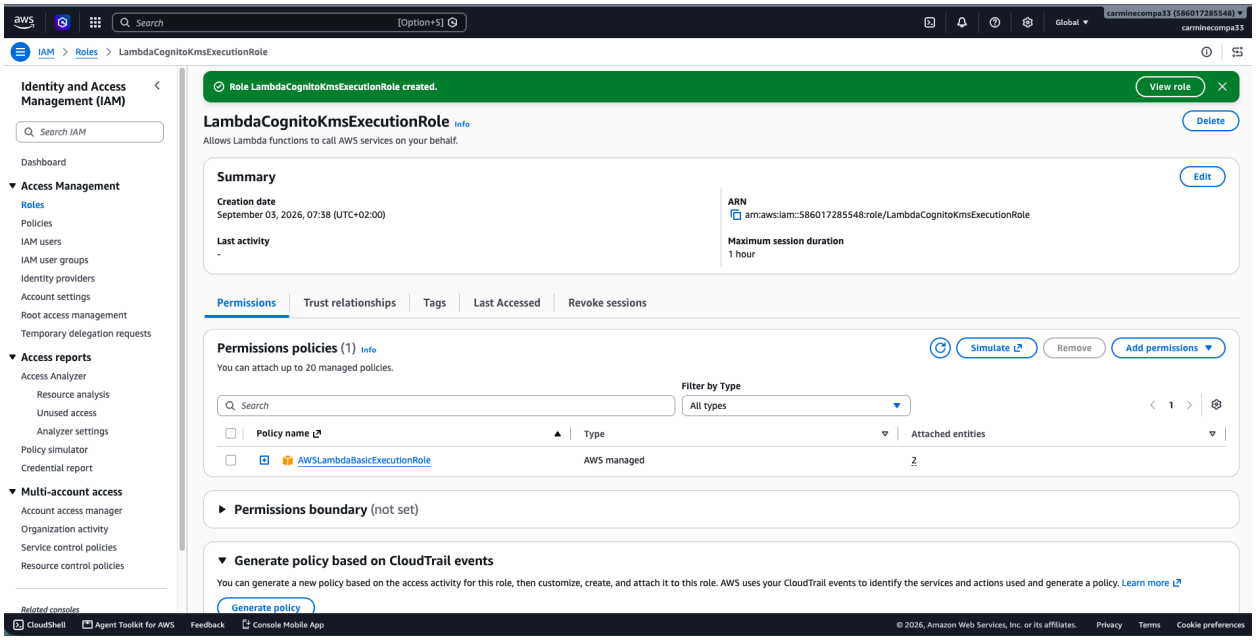
Also App client is automatically created:



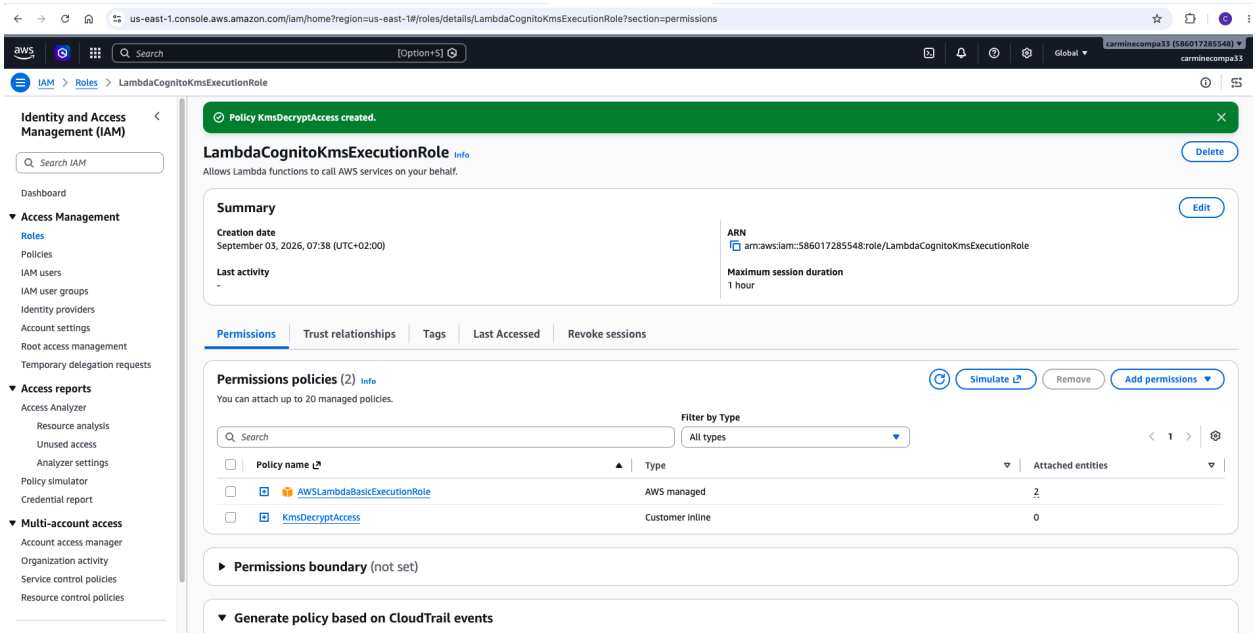
Created user in user pool with password type of login



LambdaCognitoKmsExecutionRole created with AWSLambdaBasicExecutionRole

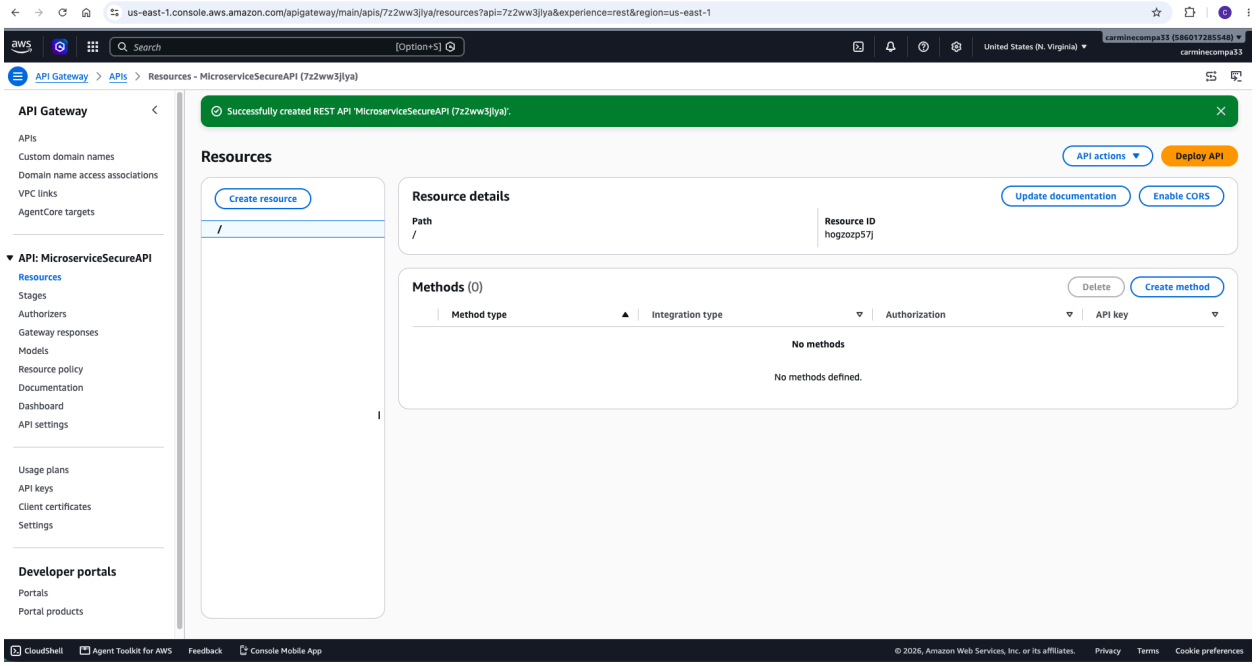


Added KmsDecryptAccess policy to the IAM role LambdaCognitoKmsExecutionRol
This gives the function minimal access required to read and write encrypted application secrets

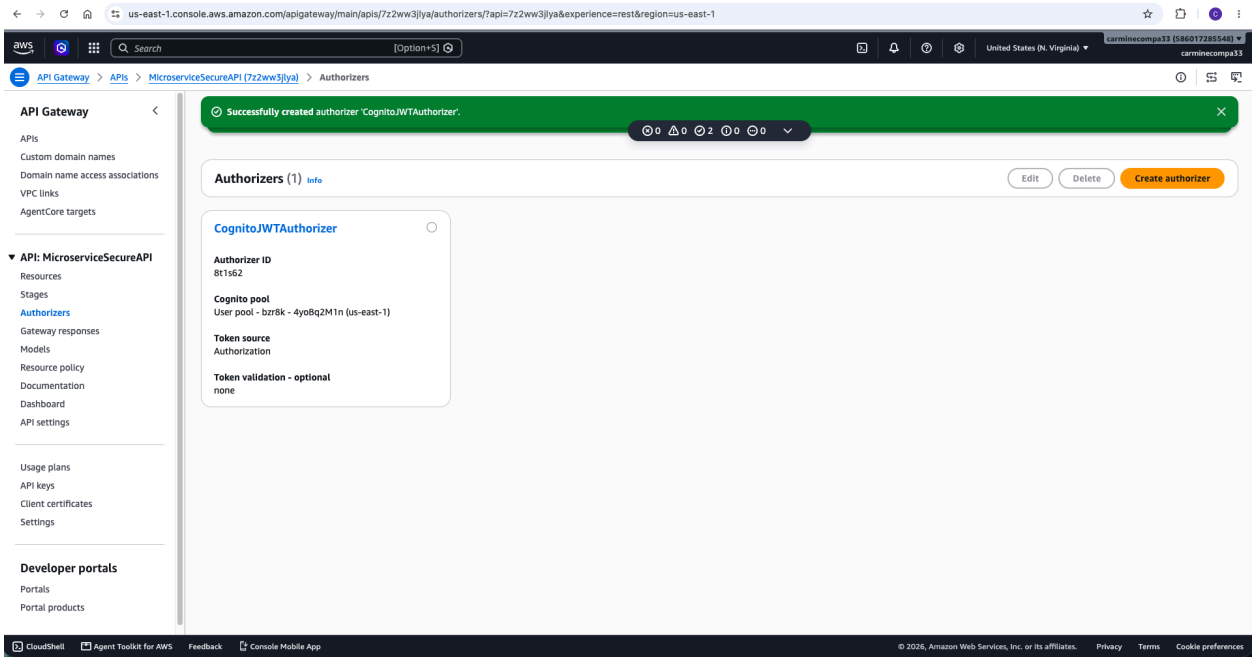


Let’s create now the Routing Layer: AWS API Gateway

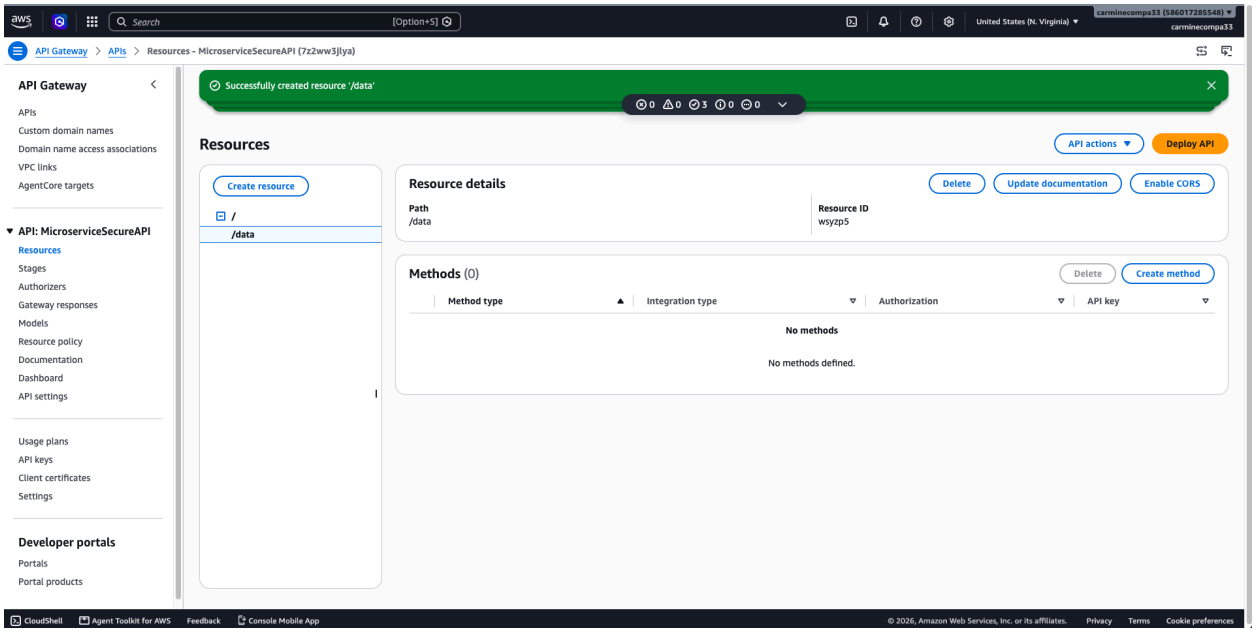
Created API Gateway - REST API



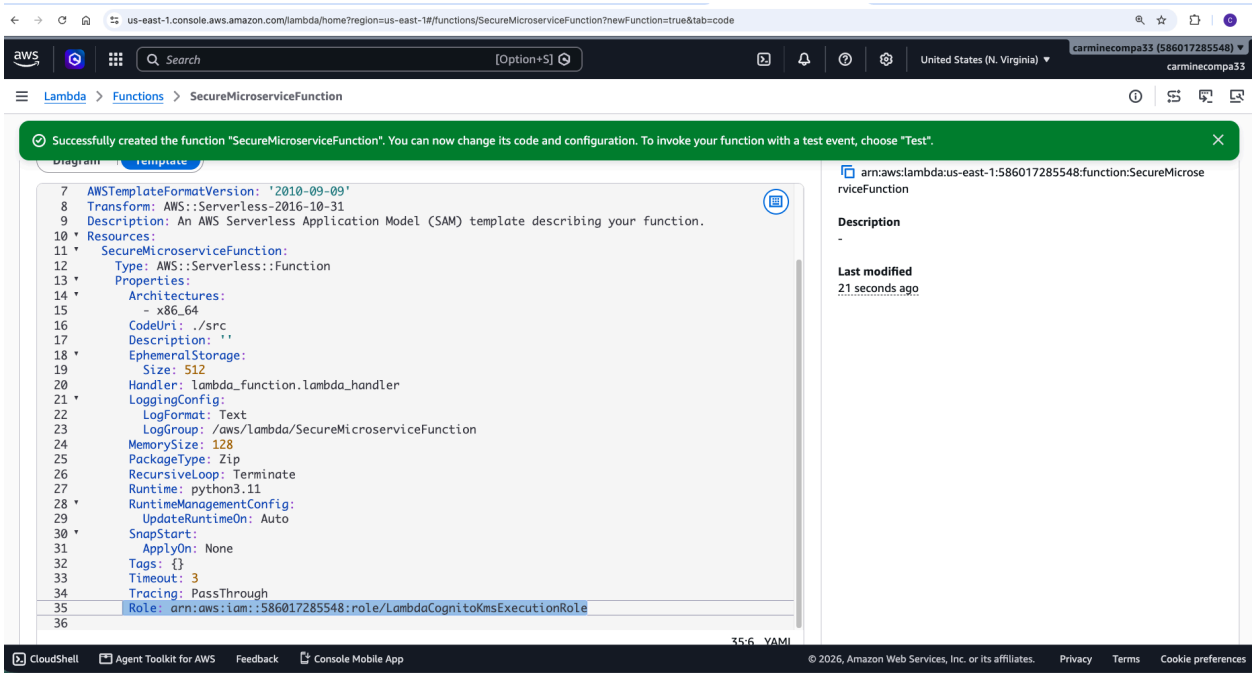
Created Authorizer with Cognito pool and Authorization as token source:



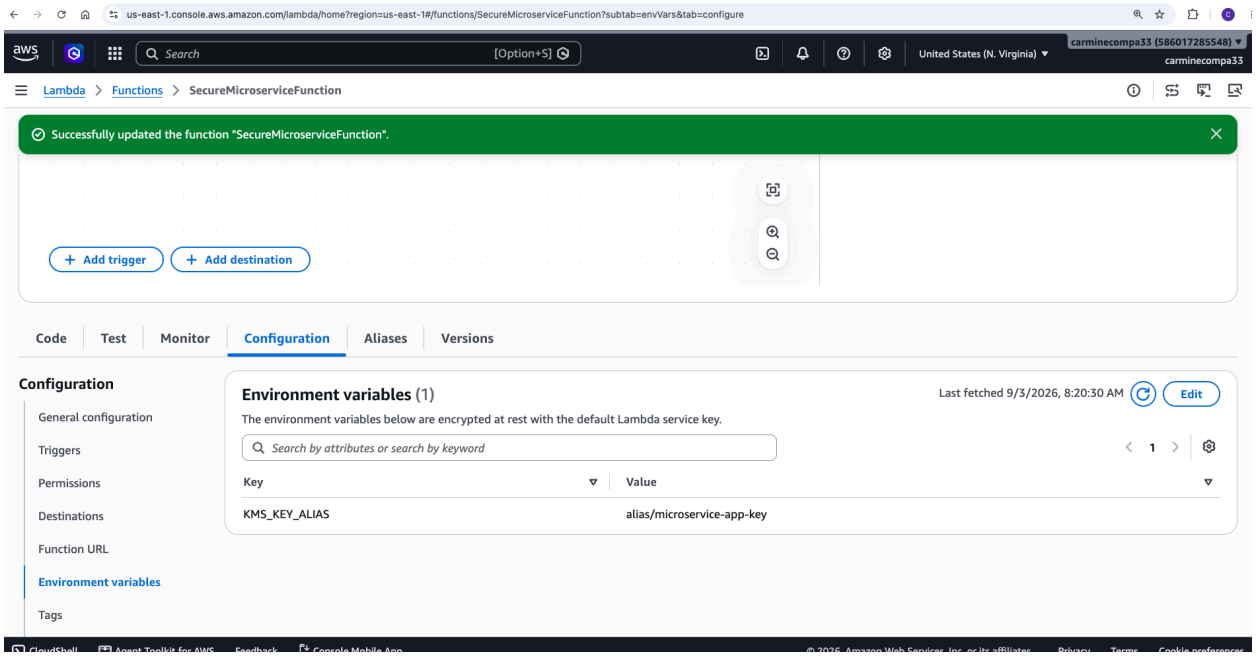
Created /data as resource



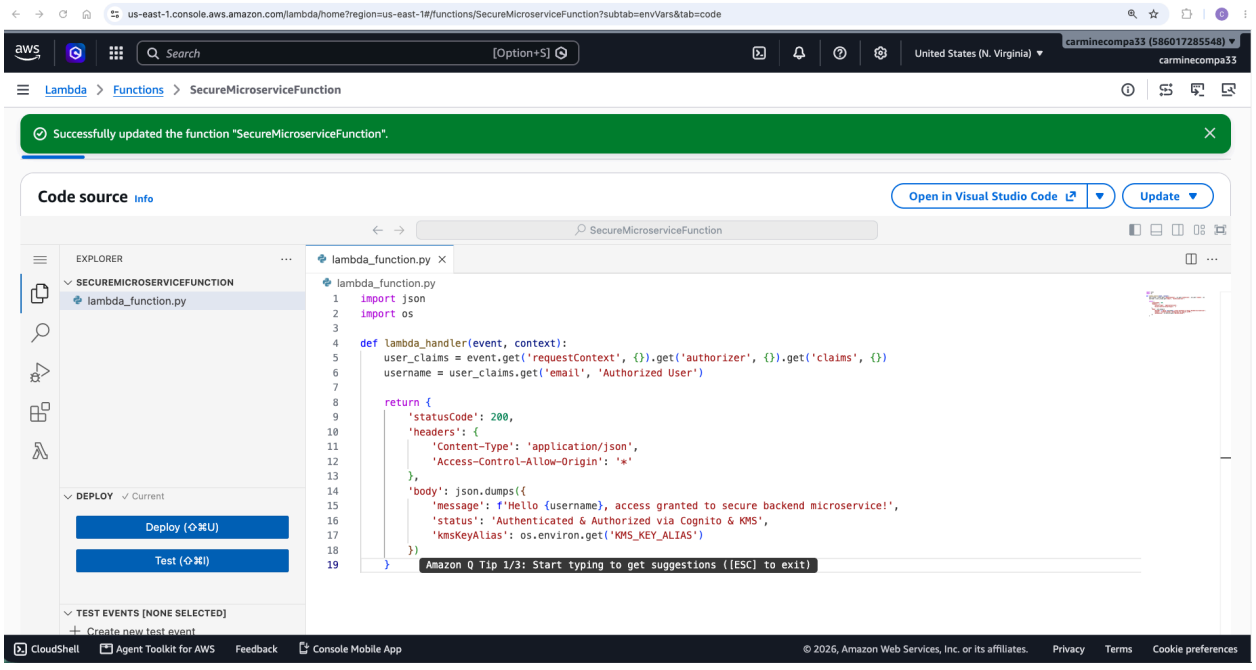
Created Lambda function with LambdaCognitoKmsExecutionRole as custom role



Added environment variable KMS_KEY_ALIAS in Lambda function

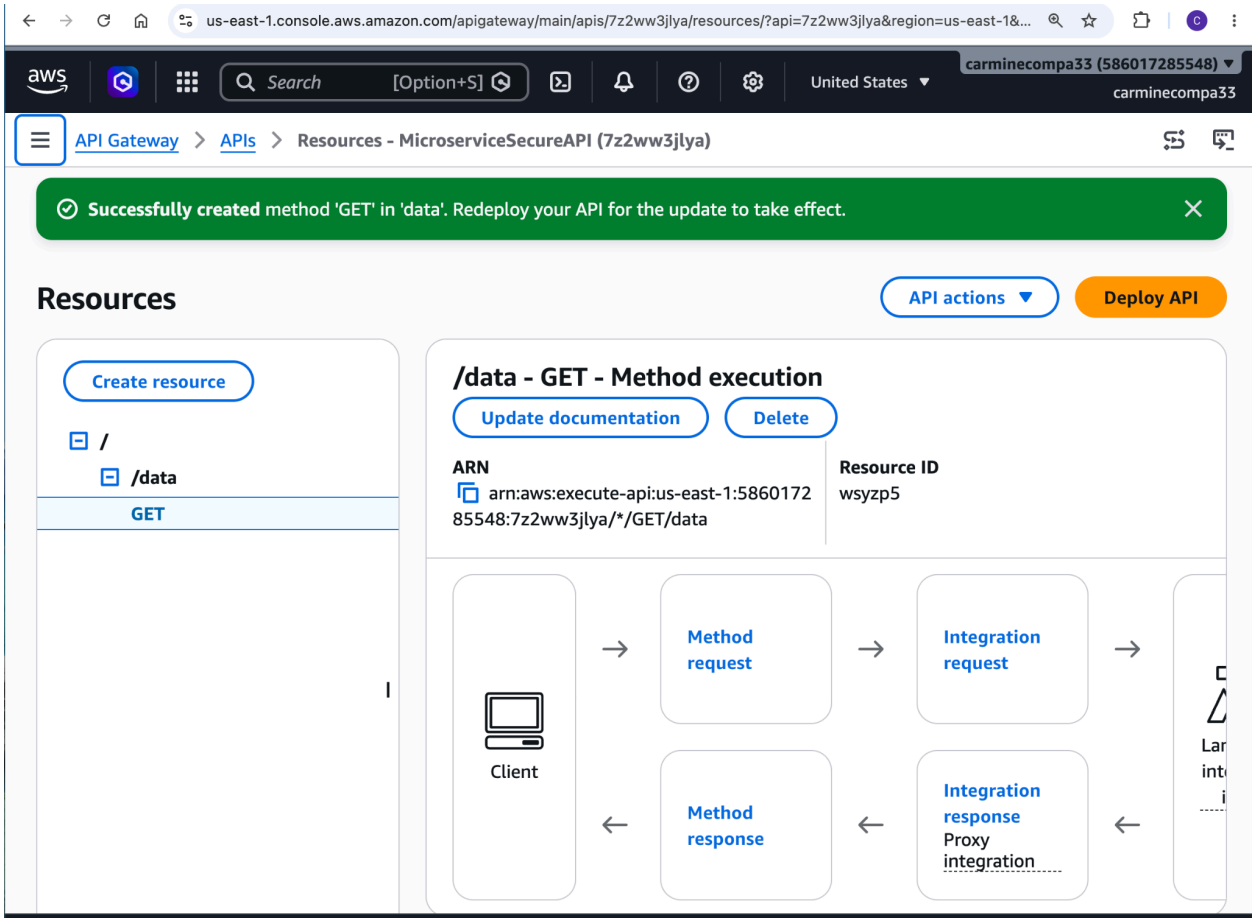


Deployed the code:

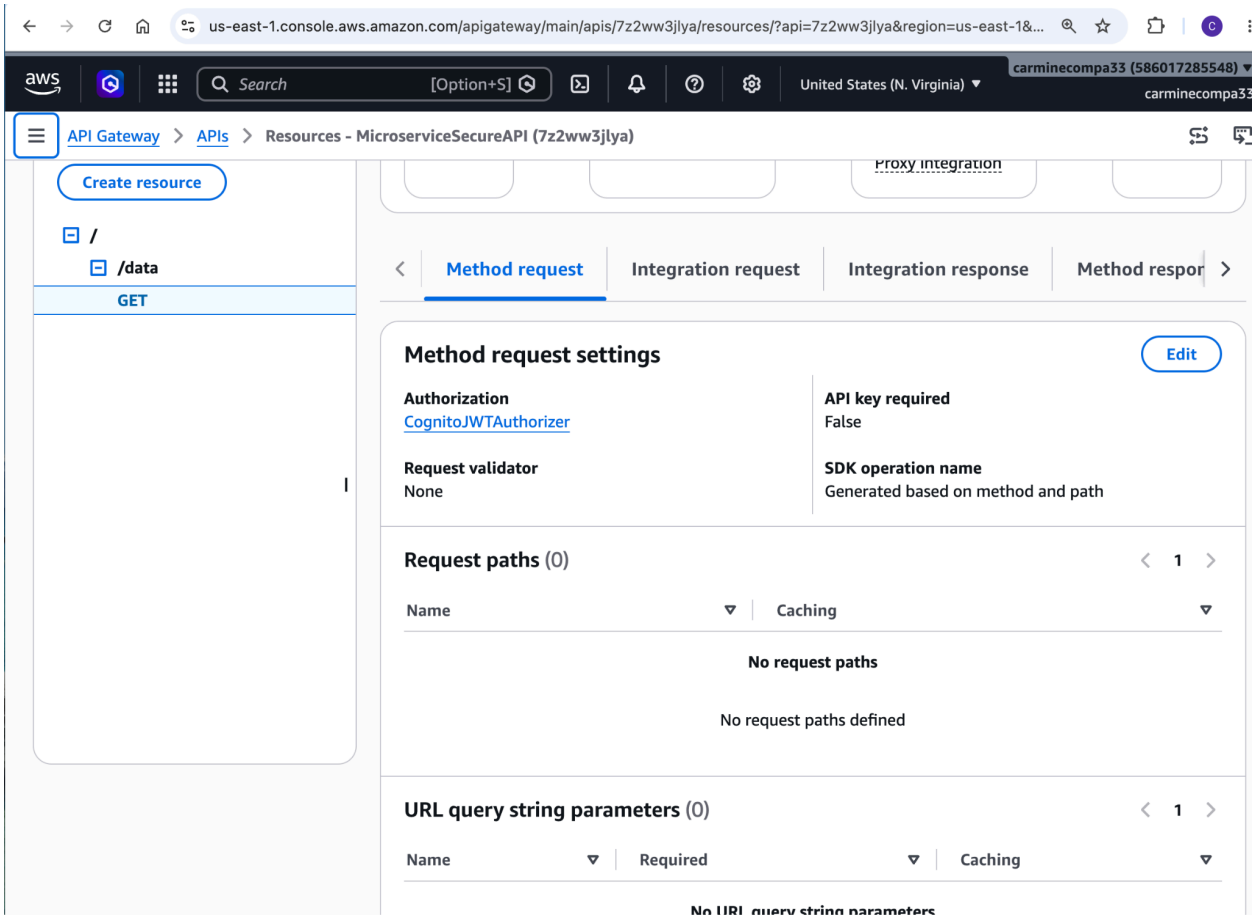


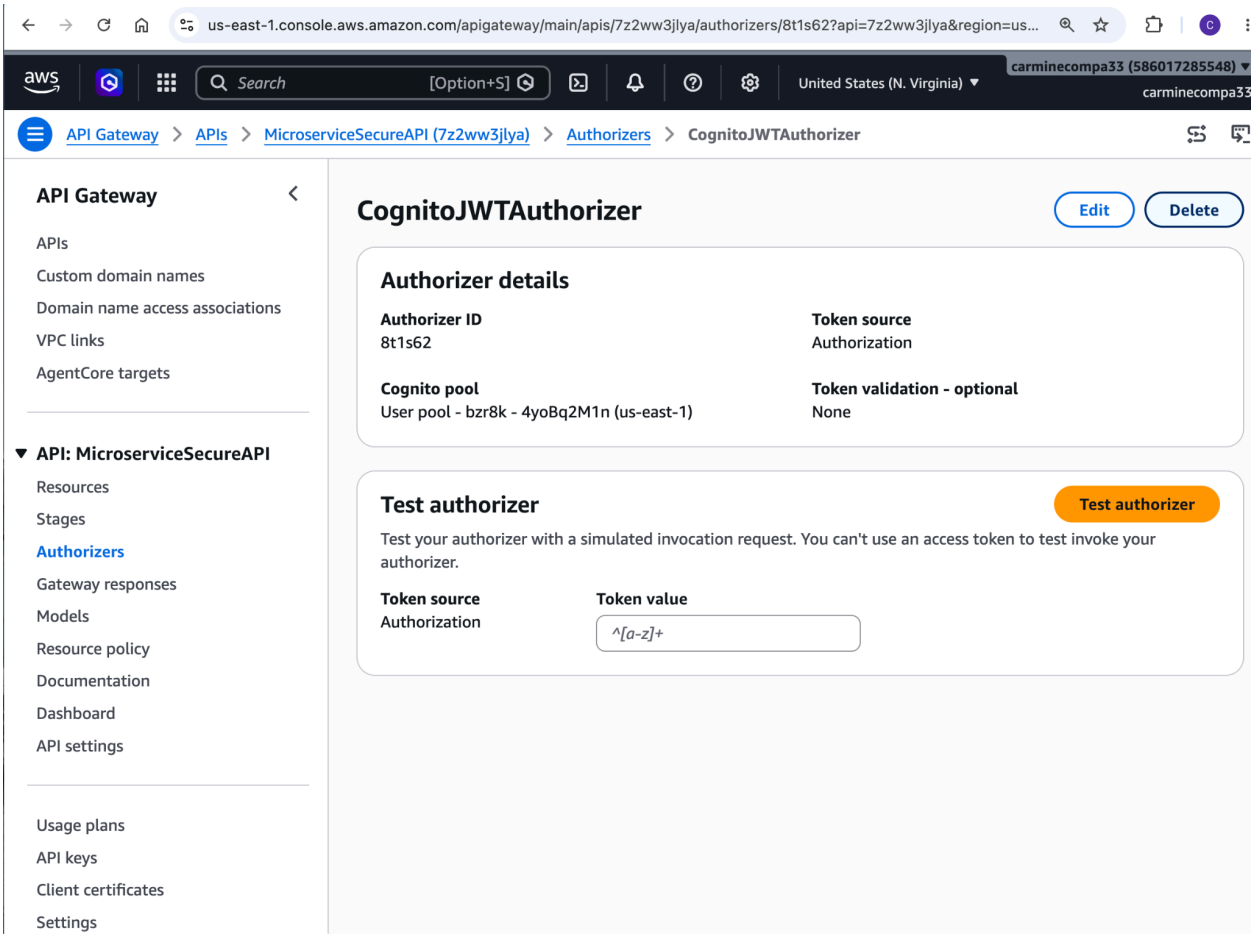
Reads Identity: It inspects the incoming request context to pull the user's verified `email` address from the Amazon Cognito JWT token claims.
Reads Environment Data: It checks the function's environment variable (`KMS_KEY_ALIAS`) to confirm which AWS KMS key alias is assigned to the microservice.
Responds Securely: It returns a JSON HTTP response (`200 OK`) greeting the user by name and confirming that they successfully passed authentication and authorization.

Created GET method in my API Gateway resource /data with proxy integration

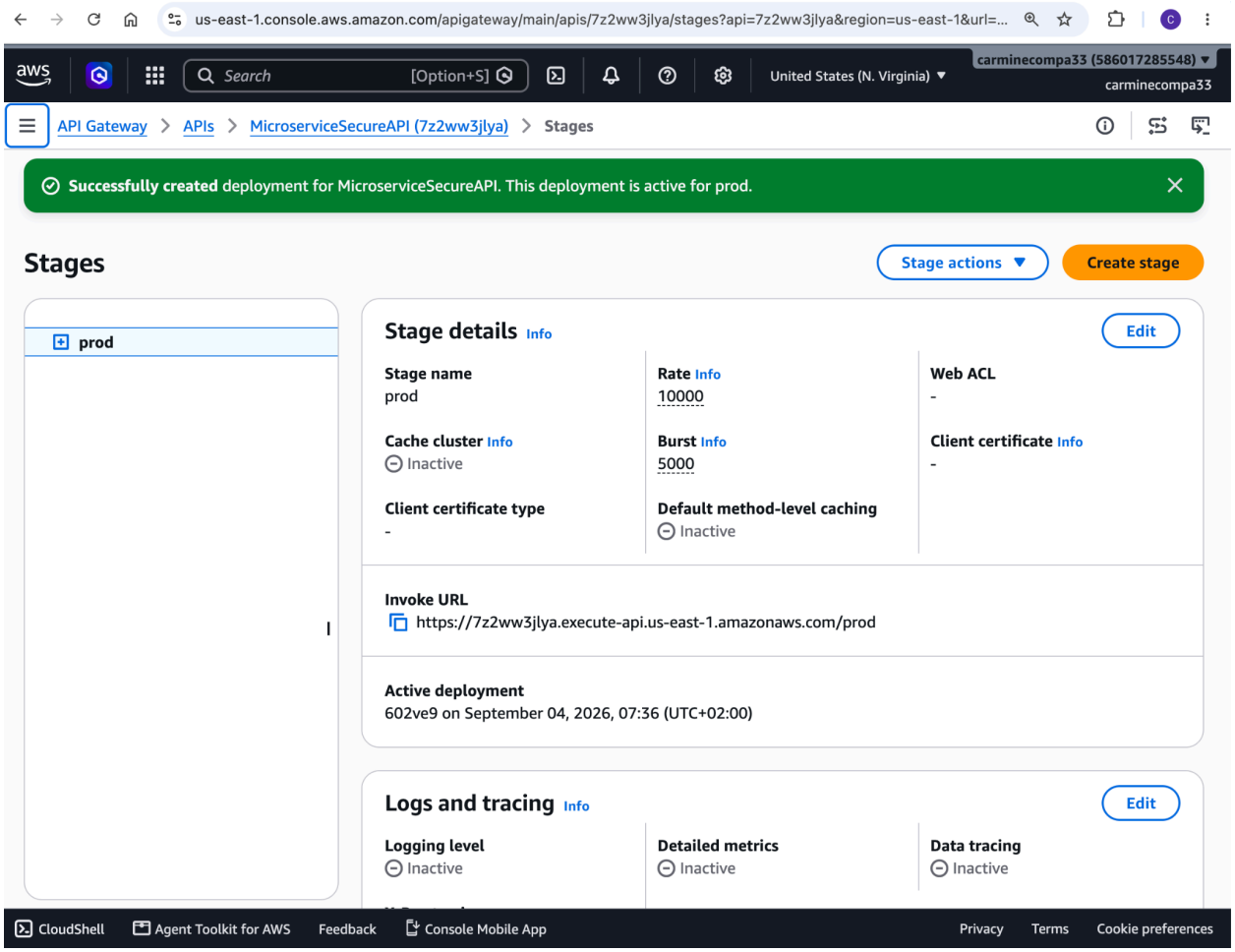


Added Cognito JWT Authorizer to the method request



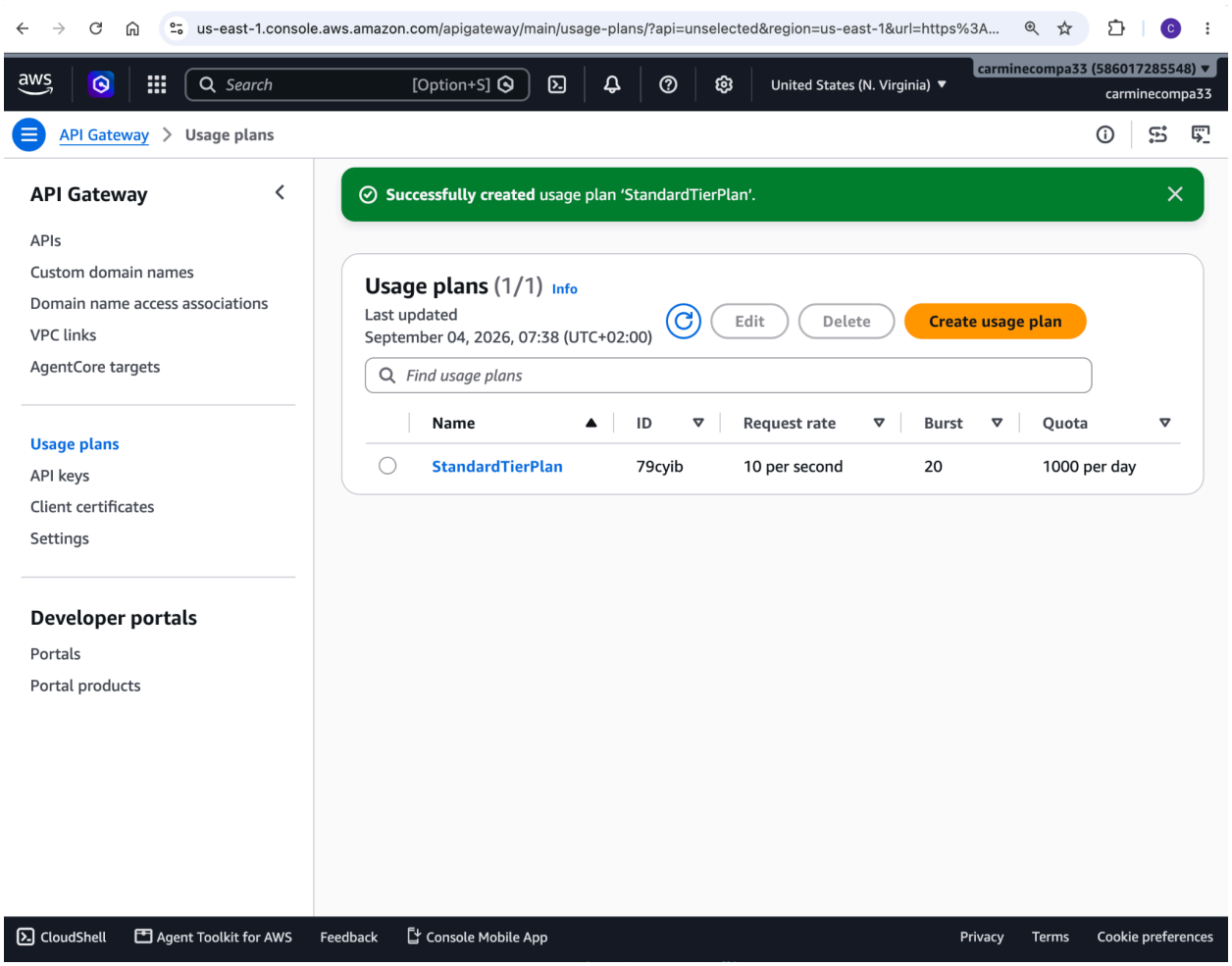

us-east-1.console.aws.amazon.com/apigateway/main/apis/7z2ww3jlya/authorizers/8t1s62?api=7z2ww3jlya&region=us...
aws
Search
[Option+S]
United States (N. Virginia)
carminecompa33 (586017285548)
carminecompa33
API Gateway
APIs
MicroserviceSecureAPI (7z2ww3jlya)
Authorizers
CognitoJWTAuthorizer
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets
API: MicroserviceSecureAPI
Resources
Stages
Authorizers
Gateway responses
Models
Resource policy
Documentation
Dashboard
API settings
Usage plans
API keys
Client certificates
Settings
CognitoJWTAuthorizer
Edit
Delete
Authorizer details
Authorizer ID
8t1s62
Token source
Authorization
Cognito pool
User pool - bzr8k - 4yoBq2M1n (us-east-1)
Token validation - optional
None
Test authorizer
Test authorizer
Test your authorizer with a simulated invocation request. You can't use an access token to test invoke your authorizer.
Token source
Authorization
Token value
^[a-z]+

Deployed API Gateway

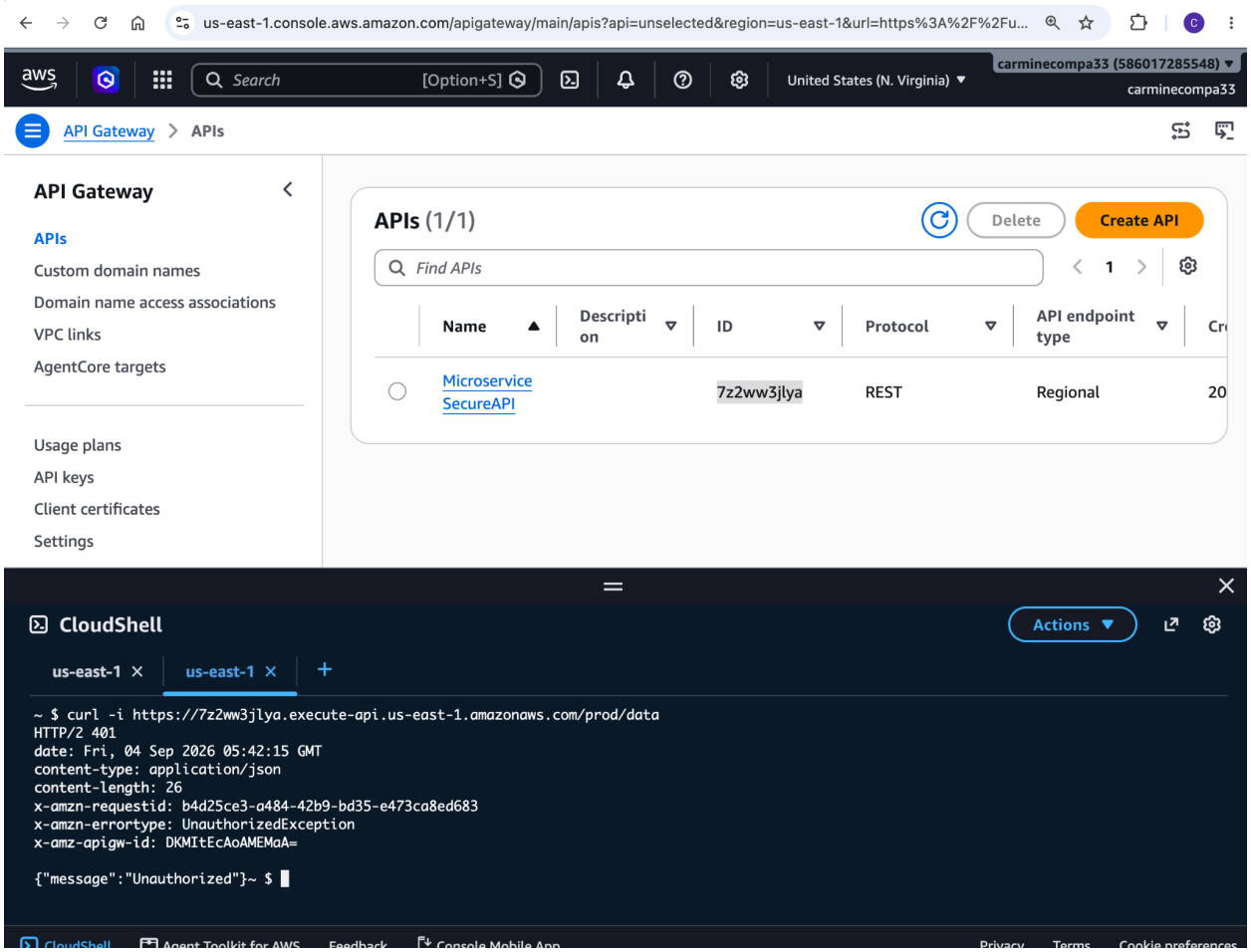


Created usage plan for the API Gateway

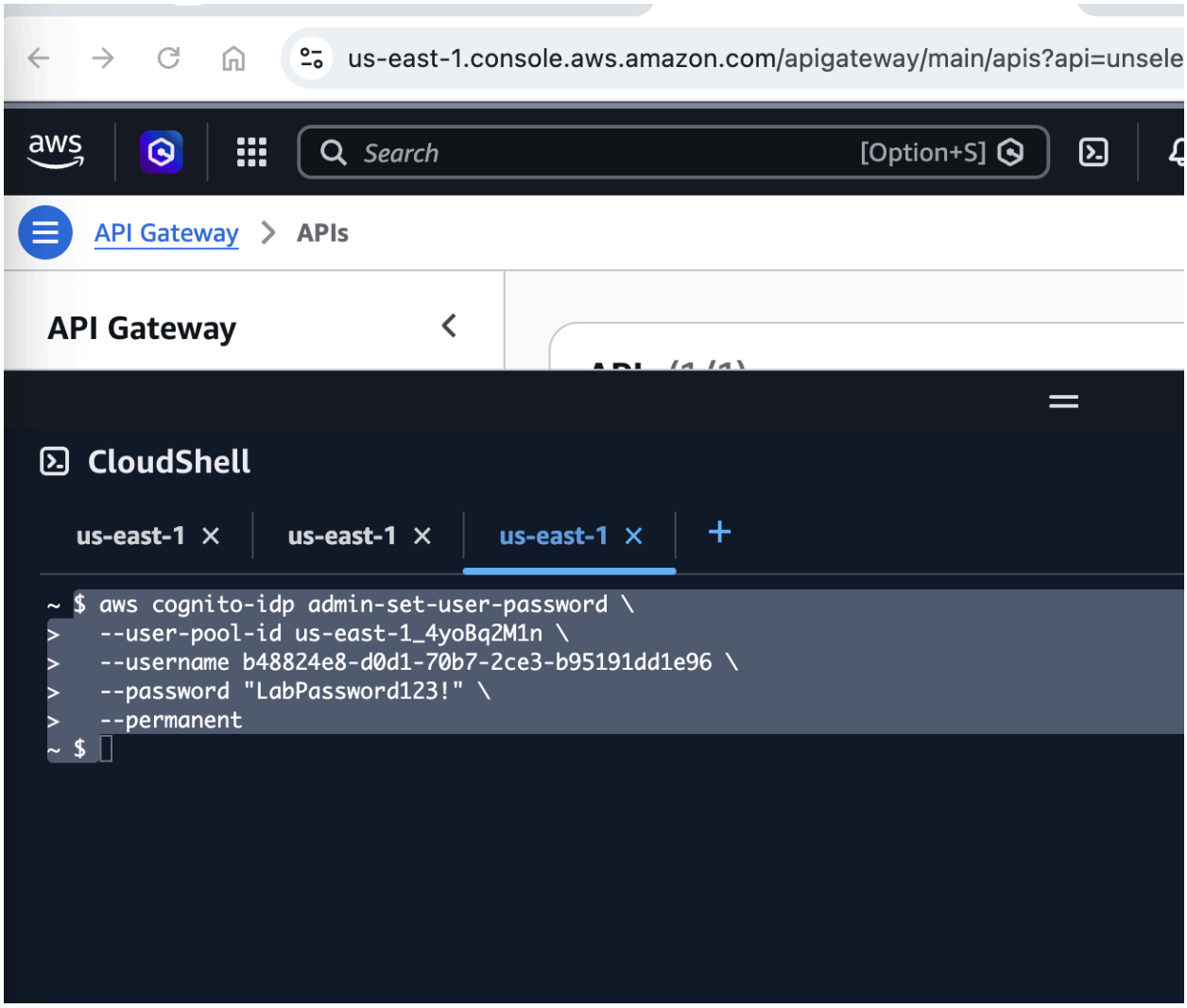
10 max requests per second
20 max concurrent requests
1000 max requests per day



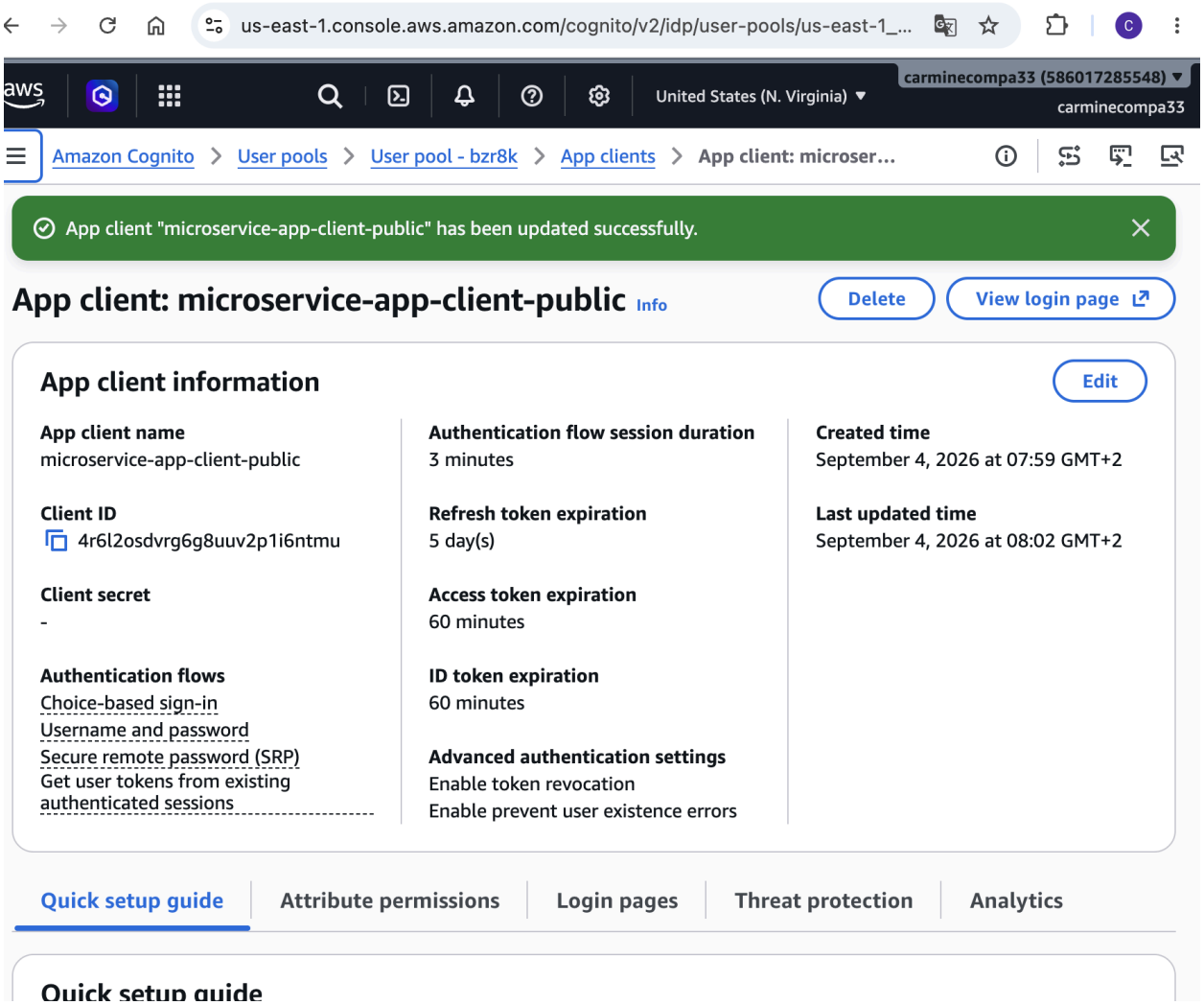
Test curl API unauthorized 401 - without JWT token



Reset password user in cognito user pool



Created new app client public



Created JWT token using the user created in the user pool with the new public app client id and with the new pwd

TOKEN=$(aws cognito-idp initiate-auth --auth-flow USER_PASSWORD_AUTH --client-id 4r6l2osdvrg6g8uuv2p1i6ntmu --auth-parameters USERNAME=carmine5544@yahoo.it,PASSWORD=LabPassword123! --query 'AuthenticationResult.IdToken' --output text)

Test curl with cognito JWT token in beared authorization header - success

